Universidad del Valle de Guatemala

Facultad de Ingeniería

CC2006 – Programación Orientada a Objetos

Segundo Ciclo, 2026

Catedrático: Kevin Antonio Velásquez Aguilar

polimorfismo_interfaces

Nombre:

Josué David García Méndez 26375

Sección # 20

Fecha de entrega: 05/11/2026

ANÁLISIS DEL SISTEMA DE INTEGRACIONES EXTERNAS
Ejercicio 6 — Polimorfismo mediante interfaces

1. Descripción del problema

Una plataforma de comercio electrónico necesita comunicarse con distintos proveedores externos para realizar pagos, consultar envíos y generar documentos. Cada proveedor ofrece capacidades diferentes: una pasarela de pago puede ejecutar cobros y reintegros, un proveedor logístico puede consultar envíos y generar guías, y un certificador FEL puede producir documentos de facturación.

El problema consiste en administrar estas integraciones mediante una estructura común, respetando las capacidades de cada servicio. El sistema también debe permitir incorporar proveedores nuevos sin modificar la lógica central que utiliza los contratos existentes.

Una situación importante ocurre cuando se solicita una operación incompatible con el proveedor seleccionado. Por ejemplo, una empresa de transporte no debe procesar un reintegro financiero. El sistema debe reconocer esa limitación, omitir la solicitud y continuar funcionando sin lanzar una excepción por esa incompatibilidad.

La solución propuesta utiliza herencia para compartir características comunes e interfaces para representar las capacidades disponibles en cada integración.

2. Objetivo general

Diseñar un sistema de administración de integraciones externas que aplique polimorfismo mediante interfaces, mantenga una sola lista de proveedores y permita utilizar sus funcionalidades desde una interfaz de consola y una interfaz gráfica desarrollada con JavaFX.

3. Objetivos específicos

- Representar las integraciones mediante una clase base común y clases concretas especializadas.
- Definir los contratos Transaccional, Documentable y Auditable.
- Utilizar interfaces como tipos de parámetros en los componentes que ejecutan las operaciones.
- Almacenar todas las integraciones en una sola lista polimórfica.
- Incorporar una carga inicial de al menos diez integraciones de distinta naturaleza.
- Permitir listar, buscar y ordenar las integraciones.

- Resolver las operaciones incompatibles sin interrumpir el funcionamiento del programa.
- Aplicar sobrecarga de métodos donde exista una diferencia útil entre sus parámetros.
- Separar las responsabilidades mediante el patrón Modelo Vista Controlador.
- Mantener coherencia entre el análisis, el diagrama UML y la implementación.

4. Alcance del sistema

El sistema será un prototipo con servicios externos simulados. Los proveedores, endpoints, estados de conexión, latencias y respuestas serán datos de demostración.

Se incluirán operaciones de consulta, cobro, reintegro, generación documental y auditoría. También se desarrollarán un menú de consola y tres vistas JavaFX.

Todas las integraciones registradas se considerarán activas durante la ejecución. Su estado de conexión será independiente de su pertenencia al catálogo. Por ello, una integración desconectada seguirá apareciendo en el listado.

Los documentos XML serán ilustrativos y no tendrán validez fiscal o legal. El prototipo no realizará conexiones reales, no manejará credenciales y no conservará información después de cerrar el programa.

Los reintegros permitirán demostrar el contrato transaccional, pero no verificarán un cobro anterior, porque el alcance no incluye contabilidad, saldos ni un historial financiero.

5. Requisitos funcionales

Código	Requisito	Comportamiento esperado
RF01	Carga inicial	Crear al menos diez integraciones de diferentes tipos al iniciar.
RF02	Catálogo único	Almacenar todas las integraciones en una sola lista polimórfica.
RF03	Listado	Mostrar identificador, nombre, tipo, proveedor, conexión y latencia.
RF04	Búsqueda	Localizar una integración por identificador y mostrar su configuración.

RF05	Ordenamiento	Ordenar las integraciones por latencia promedio mediante Comparable.
RF06	Consulta de estado	Consultar la disponibilidad general o el estado simulado de una referencia.
RF07	Cobros y reintegros	Ejecutar operaciones financieras simuladas mediante Transaccional.
RF08	Generación documental	Solicitar documentos mediante Documentable.
RF09	Auditoría	Registrar y consultar eventos asociados a las integraciones.
RF10	Operaciones incompatibles	Devolver un resultado de omisión y continuar con el proceso.
RF11	Consola	Proporcionar un menú para administrar y demostrar las funcionalidades.
RF12	Interfaz gráfica	Proporcionar tres vistas JavaFX conectadas al modelo compartido.
RF13	Extensibilidad	Incorporar proveedores compatibles sin modificar el motor de operaciones.

Las opciones de consulta, cobro, reintegro, documentos y auditoría permitirán demostrar los contratos desde ambas interfaces de usuario.

6. Organización mediante MVC

El sistema se organizará mediante el patrón Modelo Vista Controlador.

El modelo contendrá los datos, las integraciones, los contratos y las reglas de funcionamiento. También administrará el catálogo, el ordenamiento y la ejecución de operaciones.

Las vistas recogerán entradas y presentarán información. La consola mostrará un menú de texto y JavaFX utilizará formularios, tablas y áreas de resultados.

Los controladores interpretarán las solicitudes del usuario, adaptarán los datos ingresados y coordinarán las llamadas al modelo. Las reglas para cobrar, reintegrar o generar documentos permanecerán en las clases del modelo.

Esta separación permitirá utilizar las mismas reglas desde consola y JavaFX.

7. Organización de paquetes

Paquete	Responsabilidad
com.mycompany.ejercicio_6	Contener el punto de entrada y la aplicación JavaFX.
com.mycompany.ejercicio_6.modelo	Contener las integraciones, el catálogo, los resultados y los servicios.
com.mycompany.ejercicio_6.modelo.interfaces	Contener los contratos de capacidades.
com.mycompany.ejercicio_6.controlador	Coordinar las solicitudes de consola y los eventos gráficos.
com.mycompany.ejercicio_6.vista.consola	Presentar el menú y gestionar la entrada y salida de texto.
com.mycompany.ejercicio_6.vista.grafica	Identificar la organización de los recursos de presentación gráfica.

En el proyecto Maven, los archivos Java se ubicarán en src/main/java. Los archivos FXML y CSS se ubicarán en src/main/resources.

8. Interfaces del sistema

Las interfaces definirán capacidades. Cada clase implementará únicamente los contratos correspondientes a sus funciones.

8.1. Interfaz Transaccional

Representará la capacidad de ejecutar operaciones financieras simuladas.

Sus operaciones serán:

- cobrar(BigDecimal monto): ResultadoOperacion
- cobrar(BigDecimal monto, String referencia): ResultadoOperacion
- reintegrar(BigDecimal monto, String referencia): ResultadoOperacion

El método de cobro con un solo parámetro tendrá una implementación predeterminada que delegará en la variante completa utilizando la referencia SIN-REFERENCIA.

Esta sobrecarga permitirá solicitar un cobro sencillo o asociarlo con una referencia específica.

Los montos se representarán mediante BigDecimal para conservar su valor decimal sin convertirlos a double.

8.2. Interfaz Documentable

Representará la capacidad de generar documentos.

Su operación será:

- generarDocumento(String referencia): ResultadoOperacion

Un proveedor logístico generará una guía de envío y un certificador FEL generará un documento ilustrativo de facturación. Ambos responderán a través del mismo contrato.

8.3. Interfaz Auditable

Representará la capacidad de registrar y consultar eventos locales.

Sus operaciones serán:

- registrarEvento(String detalle): void
- registrarEvento(String accion, String detalle): void
- consultarAuditoria(): String

La variante de un parámetro utilizará la acción GENERAL y delegará en la variante de dos parámetros.

La auditoría será local, por lo que podrá consultarse aunque la integración esté desconectada.

8.4. Distribución de capacidades

Clase concreta	Transaccional	Documentable	Auditable
PasarelaPago	Sí	No	Sí, heredado
ProveedorLogistico	No	Sí	Sí, heredado
ServicioRastreo	No	No	Sí, heredado
CertificadorFEL	No	Sí	Sí, heredado

Comparable será una interfaz estándar de Java utilizada para el ordenamiento. No será necesario crear un archivo propio para ella.

9. Clases del modelo

9.1. IntegracionAPI

Será la clase abstracta base de todas las integraciones. Implementará Auditable y Comparable de IntegracionAPI.

Su responsabilidad será reunir los datos comunes, la configuración general, la disponibilidad, la latencia y el registro de auditoría.

Sus atributos serán:

Atributo	Tipo	Característica
identificador	String	Privado, final y obligatorio.
nombre	String	Privado, final y obligatorio.
proveedor	String	Privado, final y obligatorio.
endpoint	String	Privado, final y obligatorio.
estadoConexion	EstadoConexion	Privado y modificable mediante setter.
latenciaPromedioMs	double	Privado, finito y no negativo.
auditoria	StringBuilder	Privado y final.
FORMATO_FECHA	DateTimeFormatter	Privado, estático y final.

El constructor será protegido para que la clase únicamente se utilice como base de sus descendientes.

Sus operaciones principales serán:

- Consultar los datos mediante getters.
- Modificar el estado de conexión y la latencia mediante setters.
- Comprobar la conexión mediante isConectada().
- Obtener el tipo mediante getTipo().
- Obtener la configuración mediante getConfiguracion().
- Consultar el estado general o por referencia.
- Registrar y consultar auditoría.
- Comparar integraciones mediante compareTo().
- Producir una representación legible mediante toString().

Los métodos getTipo() y getConfiguracionEspecifica() serán abstractos para que cada descendiente proporcione su información particular.

También dispondrá de métodos de apoyo para validar texto, comprobar latencias, escapar caracteres XML y registrar los resultados de las operaciones.

9.2. PasarelaPago

Extenderá IntegracionAPI e implementará Transaccional.

Su responsabilidad será simular cobros y reintegros. Su atributo específico será medioPago, de tipo String, privado y final.

El constructor recibirá los datos comunes de la integración y el medio de pago.

Sus operaciones principales serán:

- getMedioPago()
- cobrar(BigDecimal monto, String referencia)
- reintegrar(BigDecimal monto, String referencia)
- getTipo()
- getConfiguracionEspecifica()

La variante sencilla de cobrar se heredará del contrato Transaccional.

Un método privado compartirá las validaciones de los movimientos: monto positivo, referencia obligatoria y conexión disponible. Cada resultado quedará registrado en la auditoría.

9.3. ProveedorLogistico

Extenderá IntegracionAPI e implementará Documentable.

Su responsabilidad será consultar envíos y generar guías simuladas. Su atributo específico será zonaCobertura, de tipo String, privado y final.

Sus operaciones principales serán:

- getZonaCobertura()
- consultarEstado(String referencia)
- generarDocumento(String referencia)
- getTipo()
- getConfiguracionEspecifica()

La consulta por referencia devolverá un estado simulado del envío. La generación documental producirá un XML con proveedor, referencia y cobertura.

Esta clase no implementará Transaccional, por lo que las solicitudes financieras se omitirán en el servicio de operaciones.

9.4. ServicioRastreo

Extenderá IntegracionAPI.

Su responsabilidad será consultar paquetes sin ofrecer operaciones financieras ni generación de documentos. Su atributo específico será tipoTransporte, de tipo String, privado y final.

Sus operaciones principales serán:

- getTipoTransporte()
- consultarEstado(String referencia)
- getTipo()
- getConfiguracionEspecifica()

Esta clase permitirá representar los proveedores que únicamente ofrecen consultas. Heredará la auditoría y las características comunes de IntegracionAPI.

9.5. CertificadorFEL

Extenderá IntegracionAPI e implementará Documentable.

Su responsabilidad será generar un documento XML ilustrativo de facturación. Su atributo específico será nitEmisor, de tipo String, privado y final.

Sus operaciones principales serán:

- getNitEmisor()
- generarDocumento(String referencia)
- getTipo()
- getConfiguracionEspecifica()

El documento incluirá el certificador, el NIT de demostración y la referencia recibida. Se escaparán los caracteres especiales antes de incorporarlos al XML.

9.6. ResultadoOperacion

Será una clase final e inmutable que comunicará los resultados entre el modelo, los controladores y las vistas.

Sus atributos privados y finales serán:

- estado: EstadoOperacion
- mensaje: String
- contenido: String

El atributo contenido permitirá devolver documentos, configuración o registros sin imprimirlos desde el modelo.

El constructor será privado y los objetos se crearán mediante fábricas estáticas:

- realizada(String mensaje)
- realizada(String mensaje, String contenido)
- omitida(String mensaje)
- fallida(String mensaje)

La clase también proporcionará getters, isRealizada() y toString().

9.7. CatalogoIntegraciones

Será la clase responsable de administrar la única lista de integraciones.

Su atributo principal será:

- integraciones: List<IntegracionAPI>

La colección será privada y final, y se inicializará mediante ArrayList.

Sus operaciones serán:

- agregar(IntegracionAPI integracion): ResultadoOperacion
- buscarPorIdentificador(String identificador): IntegracionAPI
- listarIntegraciones(): List<IntegracionAPI>
- getCantidad(): int
- ordenarPorLatencia(): ResultadoOperacion

El registro rechazará objetos nulos e identificadores duplicados. La búsqueda ignorará mayúsculas y espacios externos.

El listado devolverá una envoltura de lectura sobre la misma colección. El ordenamiento modificará la lista original utilizando Comparable.

9.8. ServicioOperaciones

Será el motor que coordine las solicitudes sobre las integraciones.

Tendrá un atributo privado y final:

- catalogo: CatalogoIntegraciones

Recibirá ese catálogo mediante su constructor.

Sus operaciones públicas permitirán:

- Cobrar con monto y con monto más referencia.
- Reintegrar un monto asociado con una referencia.
- Generar documentos.
- Consultar el estado general o por referencia.
- Consultar configuración.
- Consultar auditoría.

Los identificadores y referencias serán String; los montos serán BigDecimal. Todas las operaciones públicas devolverán ResultadoOperacion.

Para ejecutar capacidades utilizará métodos privados cuyos parámetros serán interfaces:

- Transaccional para procesar cobros y reintegros.
- Documentable para solicitar documentos.
- Auditable para consultar registros y registrar omisiones.

El servicio comprobará capacidades, no nombres de proveedores. Si una integración no ofrece el contrato necesario, devolverá un resultado OMITIDA.

9.9. CargadorIntegraciones

Será una clase de utilidad encargada de construir el catálogo inicial.

Tendrá un constructor privado y la operación pública estática:

- crearCatalogoInicial(): CatalogoIntegraciones

También utilizará un método privado para registrar las integraciones y detectar errores en los datos iniciales.

La distribución inicial será:

Tipo	Cantidad
Pasarelas de pago	3

Proveedores logísticos	2
Servicios de rastreo	2
Certificadores FEL	3
Total	10

Se utilizarán diferentes latencias y estados de conexión para demostrar el ordenamiento y los fallos controlados.

10. Enumeraciones

10.1. EstadoConexion

Definirá los valores:

- CONECTADA
- DESCONECTADA

Representará la disponibilidad simulada del proveedor. No determinará si la integración pertenece al catálogo.

10.2. EstadoOperacion

Definirá los valores:

- REALIZADA: la operación simulada se completó.
- OMITIDA: la integración no ofrece la capacidad solicitada.
- FALLIDA: la solicitud no pudo completarse.

Esta distinción evitará presentar una operación incompatible como si hubiera sido realizada.

11. Controladores y vistas de consola

11.1. ControladorConsola

Coordinará las solicitudes de la consola con el modelo.

Sus atributos privados y finales serán:

- catalogo: CatalogoIntegraciones
- servicio: ServicioOperaciones

El constructor recibirá ambos componentes asociados al mismo modelo.

Sus operaciones permitirán listar, buscar, ordenar, consultar estados, cobrar, reintegrar, generar documentos y consultar auditoría.

Los montos escritos por el usuario se convertirán directamente a BigDecimal. El controlador comprobará el formato del texto; la validación de monto positivo permanecerá en la pasarela de pago.

Se aceptará notación decimal común, como 150.50. Se rechazarán letras, entradas vacías, comas, separadores de miles y exponentes.

Al conectar la vista, se incorporará una operación iniciar(VistaConsola vista) para coordinar el ciclo del menú. El controlador solicitará datos mediante la vista y finalizará al seleccionar salir o al agotarse la entrada.

11.2. VistaConsola

Será responsable de mostrar información y recoger entradas de texto.

Sus atributos previstos serán:

- entrada: Scanner
- salida: PrintStream

Sus operaciones principales serán:

- mostrarMenu()
- leerTexto(String indicacion)
- mostrarIntegraciones(List<IntegracionAPI> integraciones)
- mostrarResultado(ResultadoOperacion resultado)
- mostrarMensaje(String mensaje)

La vista recorrerá la lista recibida sin crear otro catálogo. La lectura deberá reconocer el final de la entrada para permitir una salida controlada.

El menú incluirá:

1. Listar integraciones.
2. Buscar por identificador.
3. Ordenar por latencia.
4. Consultar estado.
5. Simular cobro.

6. Simular reintegro.
7. Generar documento.
8. Consultar auditoría.
9. Salir.

12. Interfaz gráfica JavaFX

La presentación gráfica se organizará en tres vistas dentro de una ventana principal.

12.1. Vista del catálogo

Se definirá en catalogo.fxml.

Mostrará una tabla con identificador, nombre, proveedor, tipo, conexión y latencia. Permitirá buscar por identificador, ordenar y abrir el detalle o las operaciones de la integración seleccionada.

La búsqueda podrá seleccionar la fila correspondiente, evitando mantener una lista independiente de resultados.

12.2. Vista de detalle

Se definirá en detalle-integracion.fxml.

Mostrará configuración y auditoría mediante dos pestañas dentro de la misma vista. Su propósito será consultar la integración seleccionada.

12.3. Vista de operaciones

Se definirá en operaciones.fxml.

Permitirá seleccionar la operación, ingresar monto y referencia cuando corresponda, y presentar el resultado.

Las operaciones incompatibles deberán poder demostrarse y mostrarse como OMITIDA. La decisión sobre la capacidad disponible permanecerá en ServicioOperaciones.

12.4. CatalogoController

Coordinará los eventos de la vista del catálogo.

Sus atributos previstos incluirán:

- Referencias al catálogo y al servicio.
- Una TableView de IntegracionAPI.

- Un campo de búsqueda.
- Una etiqueta de estado.

Sus operaciones permitirán configurar el modelo, actualizar la tabla, buscar, ordenar y abrir las otras vistas.

La tabla utilizará una envoltura respaldada por la lista del catálogo. El ordenamiento se solicitará al modelo y después se actualizará explícitamente la presentación.

12.5. DetalleIntegracionController

Coordinará la presentación de configuración y auditoría.

Sus atributos previstos incluirán el servicio, el identificador seleccionado y las áreas de texto correspondientes.

Sus operaciones permitirán configurar la selección, cargar el detalle, actualizar la auditoría y regresar al catálogo.

12.6. OperacionesController

Coordinará los formularios de operaciones.

Sus atributos previstos incluirán el servicio, el identificador seleccionado, un selector de operación, los campos de monto y referencia y un área de resultados.

Sus operaciones permitirán configurar la integración, actualizar los campos visibles, ejecutar la solicitud y regresar.

La conversión de montos seguirá los mismos criterios de entrada utilizados en consola.

12.7. Recursos gráficos

Los recursos serán:

- catalogo.fxml
- detalle-integracion.fxml
- operaciones.fxml
- estilos.css

El CSS compartirá los estilos entre las tres vistas. Los avisos sencillos podrán utilizar diálogos de JavaFX sin requerir otro archivo FXML.

13. Clases de arranque

13.1. Ejercicio_6

Será el punto de entrada general mediante main(String[] args).

Seleccionará el modo de ejecución. En consola creará una sola vez el catálogo, el servicio, el controlador y la vista.

En el modo gráfico delegará el arranque a Aplicacion, evitando cargar previamente otro catálogo.

13.2. Aplicacion

Extenderá javafx.application.Application.

Sus atributos previstos serán el catálogo y el servicio de operaciones.

La operación start(Stage escenario) creará el modelo del modo gráfico una sola vez, cargará la vista principal y entregará las referencias compartidas a sus controladores.

Las vistas gráficas trabajarán sobre ese mismo catálogo durante la ejecución.

14. Aplicación de herencia y polimorfismo

La herencia permitirá compartir configuración, conexión, latencia y auditoría mediante IntegracionAPI.

El polimorfismo se aplicará en dos niveles.

Primero, la lista almacenará referencias de tipo IntegracionAPI, aunque los objetos pertenezcan a clases concretas diferentes. Las llamadas a operaciones sobrescritas utilizarán el comportamiento de cada objeto.

Segundo, el motor recibirá capacidades mediante interfaces. Una operación que trabaje con Documentable podrá solicitar un documento tanto a un proveedor logístico como a un certificador FEL.

La incorporación de un proveedor con comportamiento existente requerirá crear y registrar otra instancia. Si necesita un comportamiento distinto, se desarrollará otra implementación que conserve los contratos.

15. Lista polimórfica única

CatalogoIntegraciones será propietario de la única colección que almacene proveedores.

No se mantendrán listas separadas para pagos, logística, rastreo o facturación. Los controladores y las vistas recibirán referencias a los objetos existentes.

La envoltura de lectura y una posible envoltura observable de JavaFX estarán respaldadas por la misma colección. No deberán convertirse en copias independientes del catálogo.

La auditoría textual de cada integración no será otra lista de proveedores.

16. Sobrecarga y sobrescritura

La sobrecarga se aplicará cuando un mismo comportamiento admita diferentes parámetros.

Los ejemplos principales serán:

- Cobrar con monto o con monto y referencia.
- Registrar un evento con detalle o con acción y detalle.
- Consultar un estado general o asociado con una referencia.
- Crear un resultado realizado con mensaje o con mensaje y contenido.

La sobrescritura permitirá especializar operaciones heredadas, como la consulta de un envío, la configuración específica y la identificación del tipo de integración.

También se sobrescribirá toString para obtener representaciones legibles.

17. Ordenamiento mediante Comparable

IntegracionAPI implementará Comparable de IntegracionAPI.

El criterio será:

1. Menor latencia promedio.
2. Identificador como desempate.

CatalogoIntegraciones solicitará el ordenamiento mediante Collections.sort sobre la lista original.

Si cambia una latencia, será necesario volver a ordenar. En JavaFX, el ordenamiento automático de las columnas no sustituirá la demostración del criterio definido mediante Comparable.

18. Encapsulación e identidad

Los atributos permanecerán privados. Los datos que deban consultarse tendrán getters y únicamente los valores que admitan modificación tendrán setters.

El identificador será inmutable para mantener estables las búsquedas y referencias.

La colección interna no se expondrá para agregar o eliminar elementos directamente desde las vistas.

No se redefinirá equals en esta etapa. El catálogo comprobará explícitamente la unicidad de los identificadores, ignorando mayúsculas y espacios externos.

19. Manejo de operaciones incompatibles y errores

Cuando se solicite una operación, el servicio buscará la integración y comprobará si ofrece el contrato correspondiente.

Si se solicita un reintegro a un proveedor logístico, el servicio reconocerá que no implementa Transaccional. Devolverá OMITIDA, registrará el motivo mediante Auditable y permitirá continuar.

Las solicitudes compatibles también podrán fallar por datos inválidos o falta de conexión.

Situación	Tratamiento
Identificador inexistente	Devolver FALLIDA con un mensaje informativo.
Identificador duplicado	Rechazar el registro sin modificar la colección.
Monto con formato inválido	Rechazar la entrada desde el controlador.
Monto cero o negativo	Devolver FALLIDA desde la pasarela.
Referencia obligatoria vacía	Devolver FALLIDA desde la implementación compatible.
Servicio desconectado	Impedir las operaciones que requieren conexión.
Capacidad inexistente	Devolver OMITIDA y registrar el evento.
Consulta de auditoría sin conexión	Permitirla porque el registro es local.
Latencia negativa, infinita o NaN	Rechazar la configuración.
Caracteres especiales en XML	Escaparlos antes de generar el documento.

La omisión de operaciones incompatibles se resolverá sin lanzar excepciones por esa causa. Los constructores y setters podrán rechazar configuraciones inválidas mediante excepciones, que deberán gestionarse cuando esos datos se capturen del usuario.

20. Criterios de verificación

La solución se comprobará mediante escenarios que permitan observar los requisitos.

Escenario	Resultado esperado
Iniciar el sistema	Cargar diez integraciones diversas.

Buscar un identificador registrado	Mostrar su configuración.
Buscar uno inexistente	Presentar un mensaje claro.
Ordenar el catálogo	Mostrar latencias de menor a mayor.
Cobrar mediante una pasarela conectada	Obtener REALIZADA.
Operar mediante una pasarela desconectada	Obtener FALLIDA.
Reintegrar mediante logística	Obtener OMITIDA y continuar.
Generar documentos con logística y FEL	Obtener documentos distintos mediante Documentable.
Consultar auditoría	Mostrar los eventos de la integración.
Agregar un proveedor compatible	Utilizarlo sin modificar el motor.
Utilizar consola y JavaFX	Mantener las mismas reglas y resultados del modelo.

También se verificarán las entradas inválidas del menú, la selección de integraciones en la tabla, las referencias opcionales y la conservación de los valores decimales.

21. Inventario y base para el diagrama UML

El diseño completo contempla 21 tipos Java propios:

- 16 clases, incluida una clase abstracta.
- 3 interfaces.
- 2 enumeraciones.

Se agregarán tres archivos FXML y un CSS. El conteo no incluye pruebas, pom.xml ni archivos package-info.java.

El diagrama UML deberá representar:

- La herencia de las cuatro integraciones concretas respecto de IntegracionAPI.
- La implementación de Auditable y Comparable por la clase base.
- La implementación de Transaccional por PasarelaPago.
- La implementación de Documentable por ProveedorLogistico y CertificadorFEL.
- La asociación del catálogo con sus integraciones.

- Las referencias de servicios y controladores al modelo.
- Las dependencias de las operaciones respecto de interfaces y resultados.
- Las clases y recursos de presentación previstos.

La multiplicidad del catálogo respecto de las integraciones será de cero a muchas. La carga inicial de diez objetos será una condición de arranque, no una multiplicidad fija.

La relación entre el catálogo y las integraciones se representará como asociación. El uso de una lista no implica por sí mismo composición del ciclo de vida, porque los objetos pueden existir antes de registrarse y recibir referencias desde otros componentes.

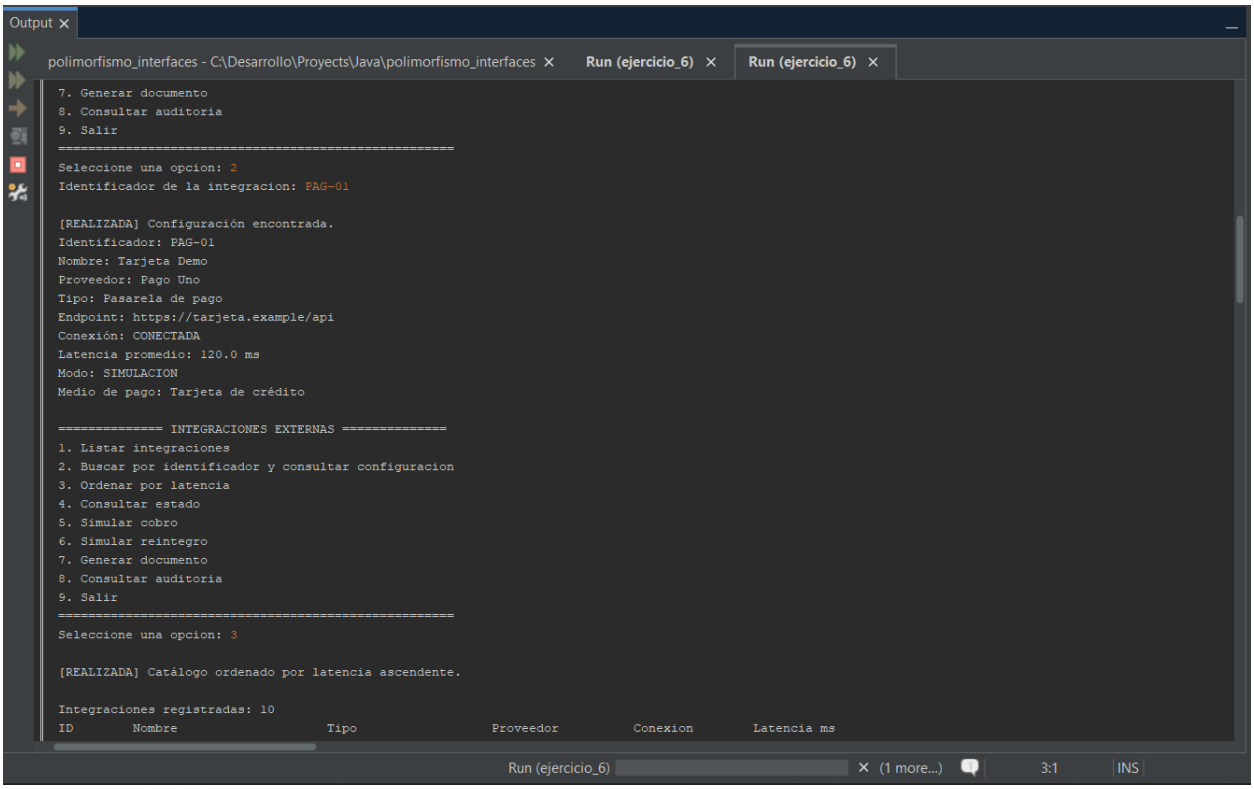
Salidas en consola:

```
Ejercicio 6 - Administracion de integraciones simuladas
Catalogo inicial: 10 integraciones.

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
================================================
Seleccione una opcion: 1

Integraciones registradas: 10
ID        Nombre                  Tipo                 Proveedor          Conexion        Latencia ms
PAG-01    Tarjeta Demo            Pasarela de pago     Pago Uno           CONECTADA            120.00
PAG-02    Transferencia Demo      Pasarela de pago     Pago Dos           CONECTADA            210.00
PAG-03    Billetera Demo          Pasarela de pago     Pago Tres          DESCONECTADA          95.00
LOG-01    Envio Nacional Demo     Proveedor logistico  Ruta Uno           CONECTADA            180.00
LOG-02    Última Milla Demo       Proveedor logístico  Ruta Dos           CONECTADA             80.00
RAS-01    Rastreo Terrestre Demo  Servicio de rastreo  Rastreo Uno        CONECTADA             60.00
RAS-02    Rastreo Aéreo Demo      Servicio de rastreo  Rastreo Dos        DESCONECTADA         250.00
FEL-01    Facturación Uno Demo    Certificador FEL     Certificador Uno   CONECTADA            140.00
FEL-02    Facturación Dos Demo    Certificador FEL     Certificador Dos   CONECTADA            120.00
FEL-03    Facturación Tres Demo   Certificador FEL     Certificador Tres  CONECTADA            300.00

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
```




```
LOG-02    Última Milla Demo         Proveedor logistico   Ruta Dos           CONECTADA          80.00
PAG-03    Billetera Demo            Pasarela de pago      Pago Tres          DESCONECTADA       95.00
FEL-02    Facturación Dos Demo      Certificador FEL      Certificador Dos   CONECTADA         120.00
PAG-01    Tarjeta Demo              Pasarela de pago      Pago Uno           CONECTADA         120.00
FEL-01    Facturación Uno Demo      Certificador FEL      Certificador Uno   CONECTADA         140.00
LOG-01    Envío Nacional Demo       Proveedor logistico   Ruta Uno           CONECTADA         180.00
PAG-02    Transferencia Demo        Pasarela de pago      Pago Dos           CONECTADA         210.00
RAS-02    Rastreo Aéreo Demo        Servicio de rastreo   Rastreo Dos        DESCONECTADA      250.00
FEL-03    Facturación Tres Demo     Certificador FEL      Certificador Tres  CONECTADA         300.00

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
=====================================================
Seleccione una opcion: 4
Identificador de la integracion: PAG-01
Referencia (Enter para estado general): primera

[REALIZADA] Servicio disponible (simulado): Tarjeta Demo. Referencia: primera

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
```

```
9. Salir
==================================================
Seleccione una opcion: 5
Identificador de la integracion: LOG-02
Monto (por ejemplo 150.50): 667
Referencia (opcional): primera

[OMITIDA] La integración no admite la operación COBRO. El proceso puede continuar.

============== INTEGRACIONES EXTERNAS ==============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: FEL-02
Opcion invalida. Seleccione un numero del 1 al 9.

============== INTEGRACIONES EXTERNAS ==============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: 5
```

```
Monto (por ejemplo 150.50): 667
Referencia (opcional): segunda]

[OMITIDA] La integración no admite la operación COBRO. El proceso puede continuar.

============== INTEGRACIONES EXTERNAS ==============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: 5
Identificador de la integracion: PAG-01
Monto (por ejemplo 150.50): 667
Referencia (opcional): segunda

[REALIZADA] COBRO SIMULADO por Tarjeta Demo. Monto: 667. Referencia: segunda

============== INTEGRACIONES EXTERNAS ==============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: 6
```

```
==================================================
Seleccione una opcion: 6
Identificador de la integracion: PAG-01
Monto (por ejemplo 150.50): 669
Referencia del reintegro: segunda

[REALIZADA] REINTEGRO SIMULADO por Tarjeta Demo. Monto: 669. Referencia: segunda

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: 7
Identificador de la integracion: LOG-01
Referencia del documento: referencial

[REALIZADA] Guía simulada generada.
<guiaEnvio simulada="true">
  <proveedor>Ruta Uno</proveedor>
  <referencia>referencial</referencia>
  <cobertura>Nacional</cobertura>
</guiaEnvio>

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
```

```
  <referencia>referencial</referencia>
  <cobertura>Nacional</cobertura>
</guiaEnvio>

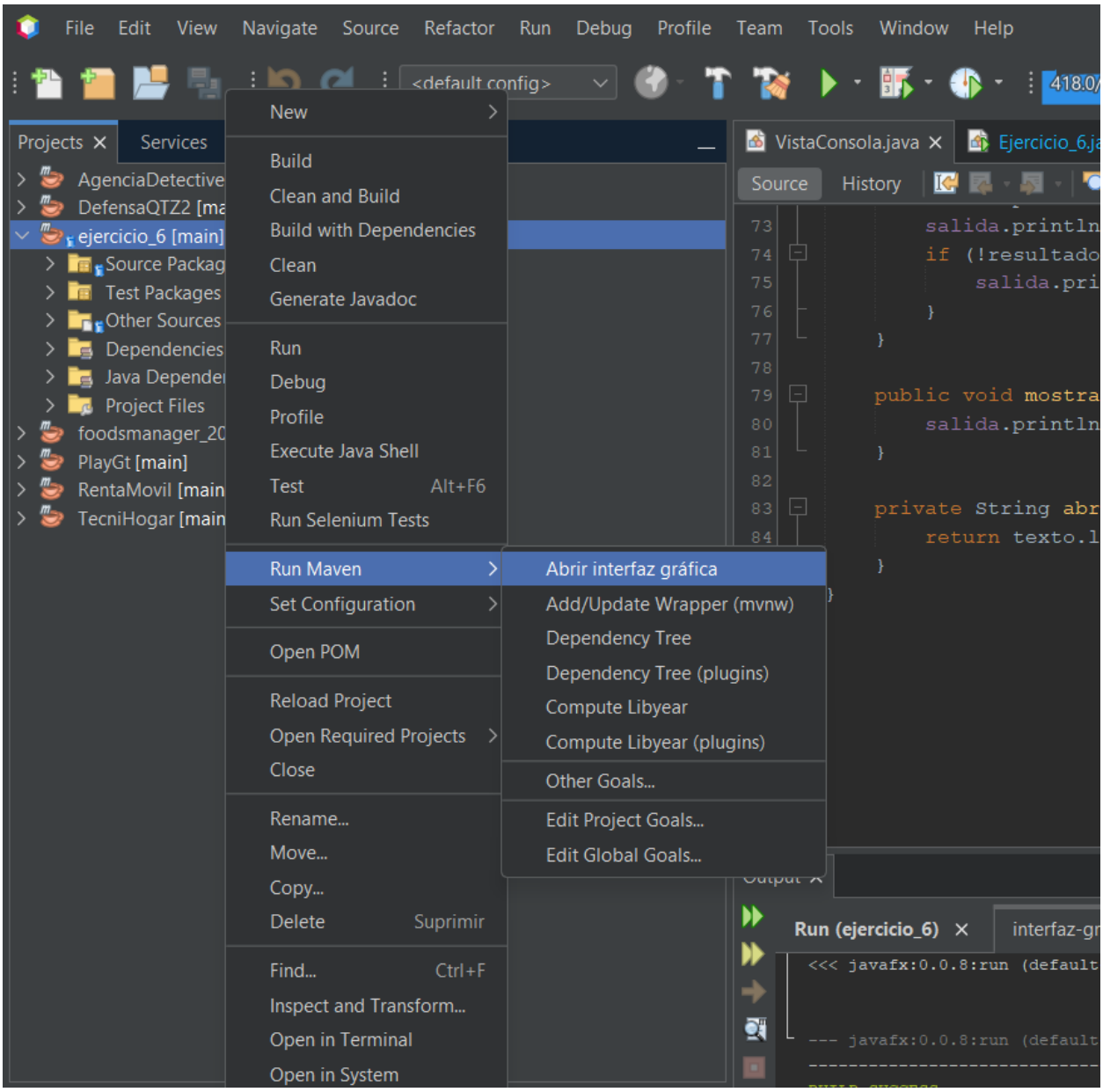
============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion: 8
Identificador de la integracion: LOG-01

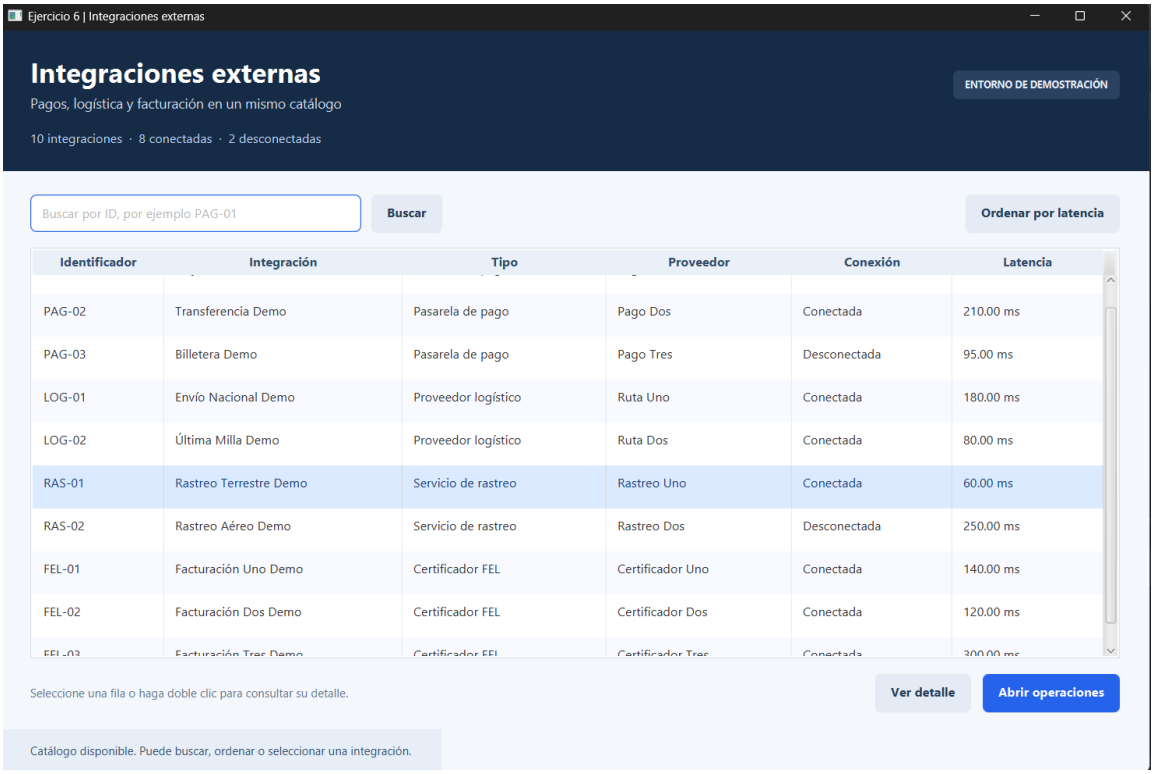
[REALIZADA] Auditoria local consultada.
2026-11-04 18:04:52 | CARGA | Integración creada en modo simulación.
2026-11-04 18:12:13 | DOCUMENTO | REALIZADA: Guía simulada generada.

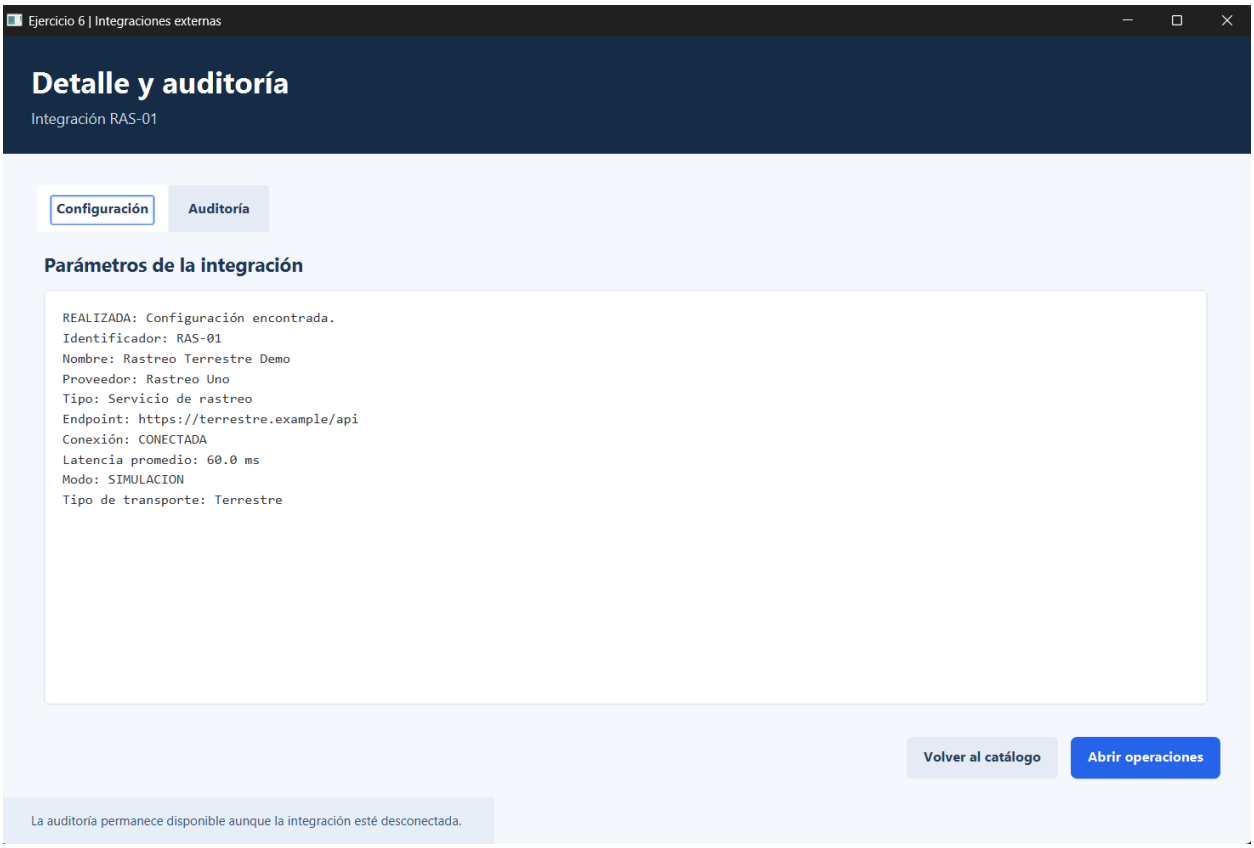

============= INTEGRACIONES EXTERNAS =============
1. Listar integraciones
2. Buscar por identificador y consultar configuracion
3. Ordenar por latencia
4. Consultar estado
5. Simular cobro
6. Simular reintegro
7. Generar documento
8. Consultar auditoria
9. Salir
==================================================
Seleccione una opcion:
```

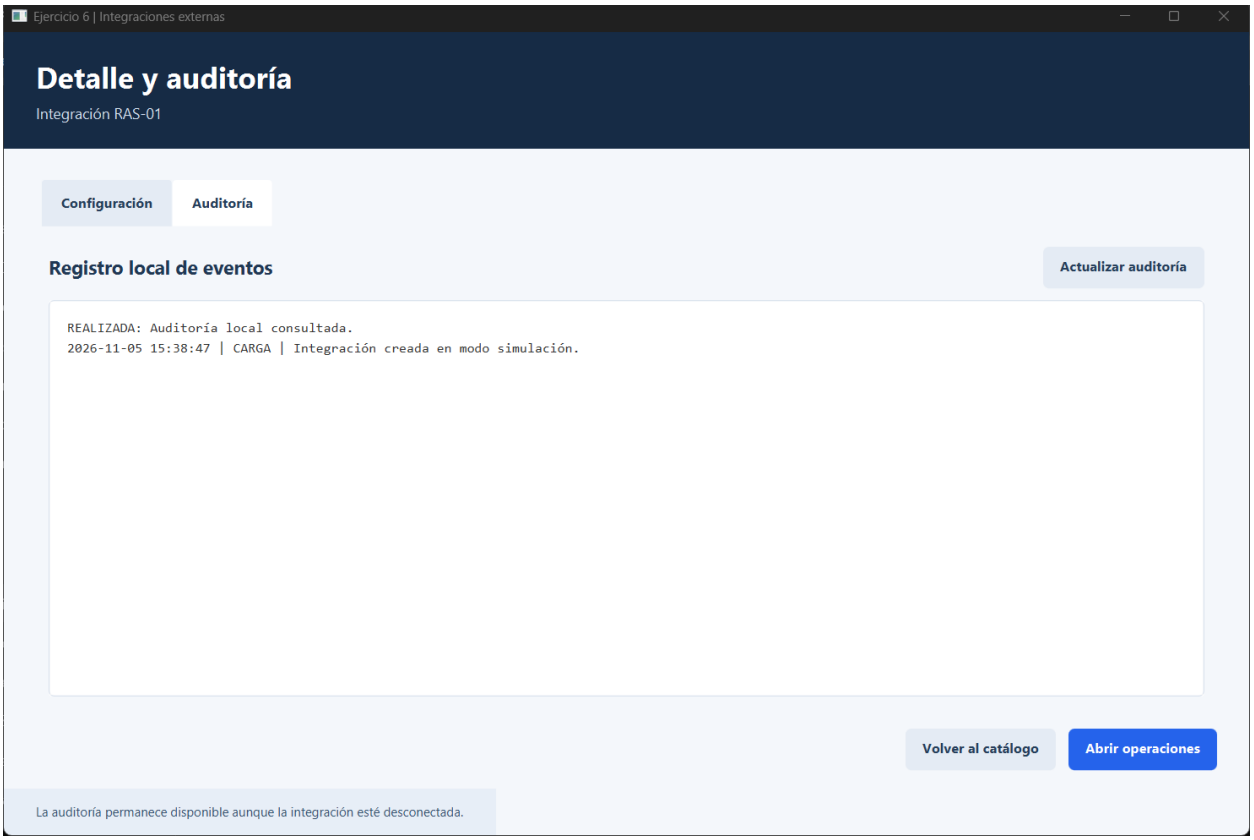
Vista desde la interfaz gráfica:

Para la interfaz gráfica, dar Click derecho en el proyecto ejercicio_6 y darle en Run Maven y abrir interfaz grafica


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Projects
Services
AgenciaDetective
DefensaQTZ2 [ma
ejercicio_6 [main]
Source Packag
Test Packages
Other Sources
Dependencies
Java Depende
Project Files
foodsmanager_20
PlayGt [main]
RentaMovil [main
TecniHogar [main
New
Build
Clean and Build
Build with Dependencies
Clean
Generate Javadoc
Run
Debug
Profile
Execute Java Shell
Test Alt+F6
Run Selenium Tests
Run Maven
Set Configuration
Open POM
Reload Project
Open Required Projects
Close
Rename...
Move...
Copy...
Delete Suprimir
Find... Ctrl+F
Inspect and Transform...
Open in Terminal
Open in System
Abrir interfaz gráfica
Add/Update Wrapper (mvnw)
Dependency Tree
Dependency Tree (plugins)
Compute Libyear
Compute Libyear (plugins)
Other Goals...
Edit Project Goals...
Edit Global Goals...
VistaConsola.java
Source
History
salida.println
if (!resultado
salida.pri
}
public void mostra
salida.println
}
private String abr
return texto.l
Run (ejercicio_6)
interfaz-gr
<<< javafx:0.0.8:run (default
--- javafx:0.0.8:run (default


Ejercicio 6 | Integraciones externas
Integraciones externas
Pagos, logística y facturación en un mismo catálogo
ENTORNO DE DEMOSTRACIÓN
10 integraciones · 8 conectadas · 2 desconectadas
Buscar por ID, por ejemplo PAG-01
Buscar
Ordenar por latencia
Identificador
Integración
Tipo
Proveedor
Conexión
Latencia
PAG-02 Transferencia Demo Pasarela de pago Pago Dos Conectada 210.00 ms
PAG-03 Billetera Demo Pasarela de pago Pago Tres Desconectada 95.00 ms
LOG-01 Envío Nacional Demo Proveedor logístico Ruta Uno Conectada 180.00 ms
LOG-02 Última Milla Demo Proveedor logístico Ruta Dos Conectada 80.00 ms
RAS-01 Rastreo Terrestre Demo Servicio de rastreo Rastreo Uno Conectada 60.00 ms
RAS-02 Rastreo Aéreo Demo Servicio de rastreo Rastreo Dos Desconectada 250.00 ms
FEL-01 Facturación Uno Demo Certificador FEL Certificador Uno Conectada 140.00 ms
FEL-02 Facturación Dos Demo Certificador FEL Certificador Dos Conectada 120.00 ms
Seleccione una fila o haga doble clic para consultar su detalle.
Ver detalle
Abrir operaciones
Catálogo disponible. Puede buscar, ordenar o seleccionar una integración.


Ejercicio 6 | Integraciones externas
Detalle y auditoría
Integración RAS-01
Configuración
Auditoría
Parámetros de la integración
REALIZADA: Configuración encontrada.
Identificador: RAS-01
Nombre: Rastreo Terrestre Demo
Proveedor: Rastreo Uno
Tipo: Servicio de rastreo
Endpoint: https://terrestre.example/api
Conexión: CONECTADA
Latencia promedio: 60.0 ms
Modo: SIMULACION
Tipo de transporte: Terrestre
Volver al catálogo
Abrir operaciones
La auditoría permanece disponible aunque la integración esté desconectada.



Ordenada por latencia:

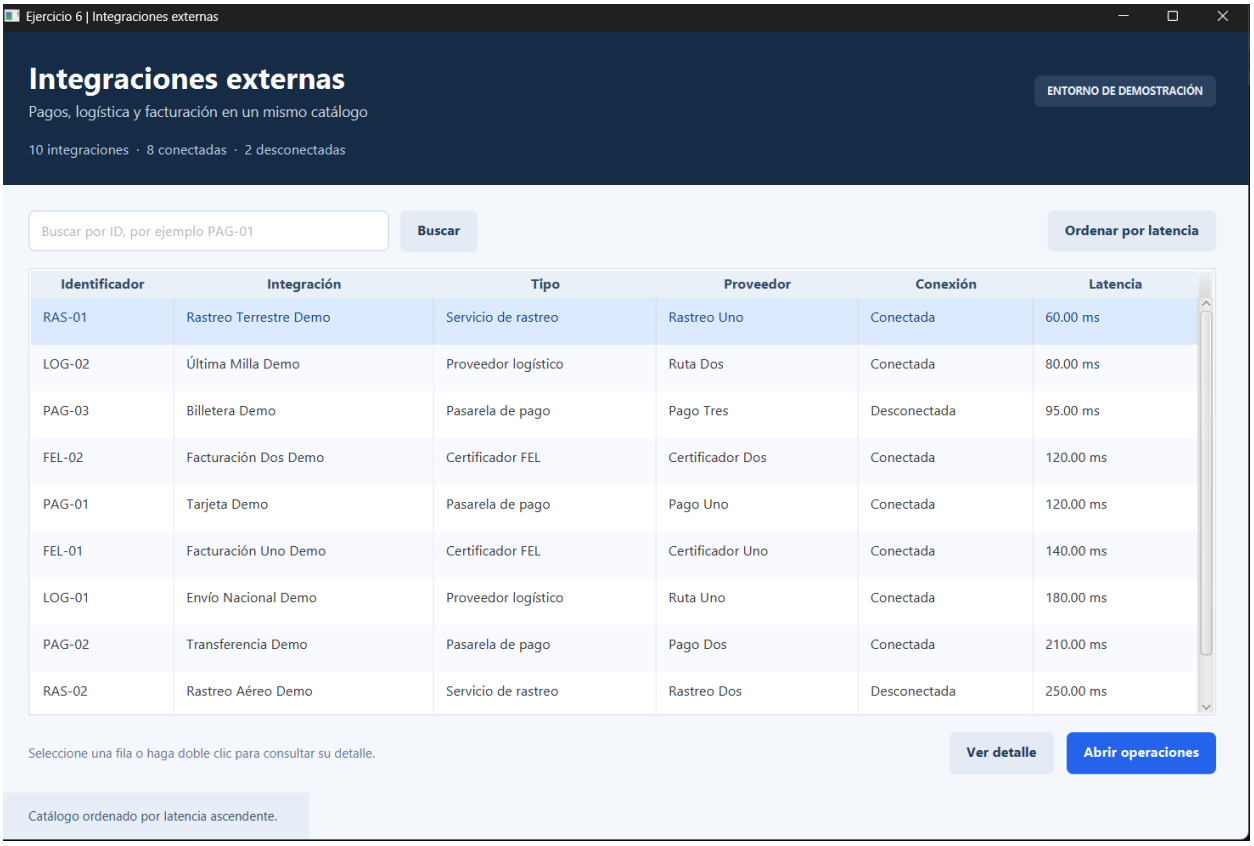
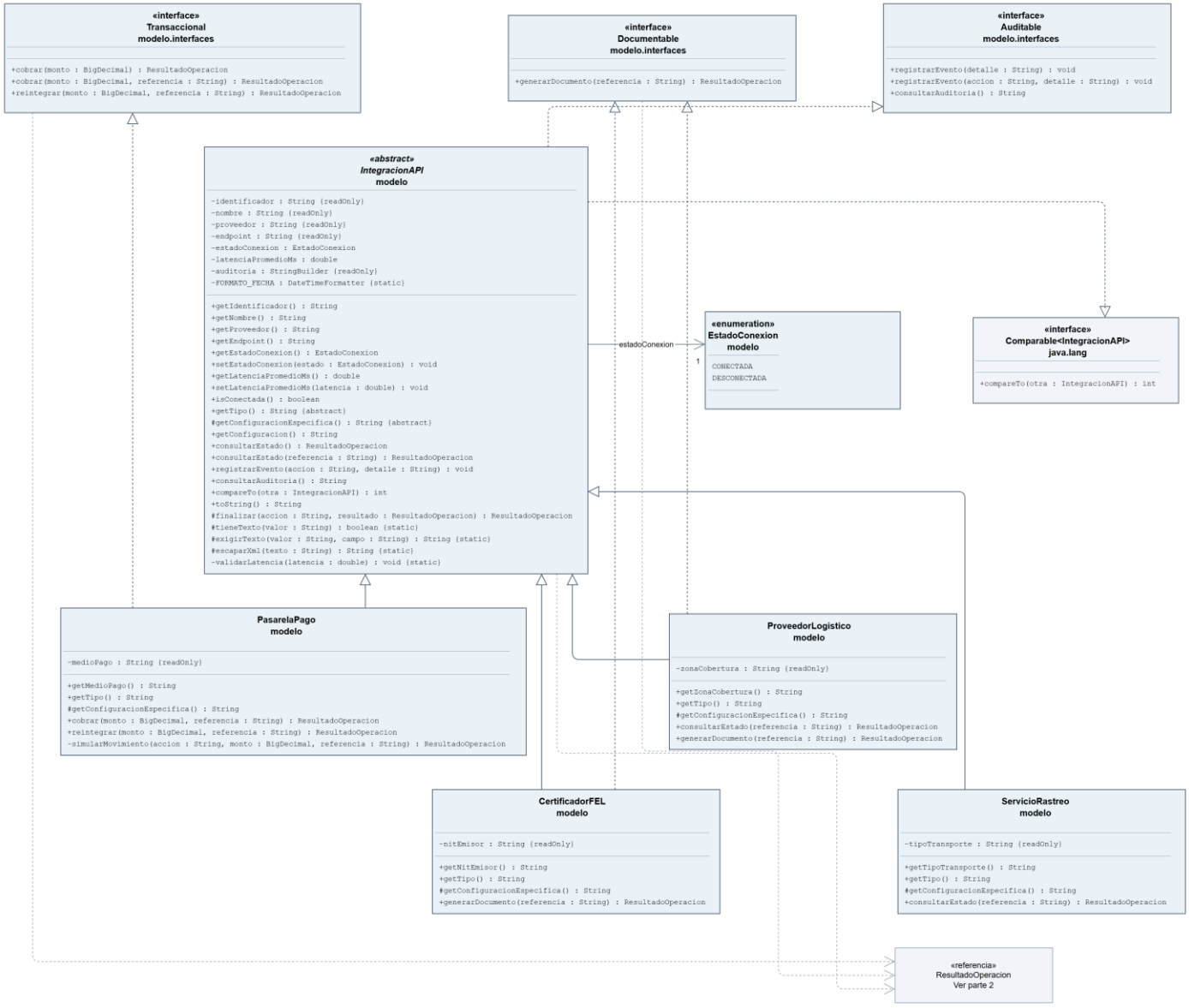


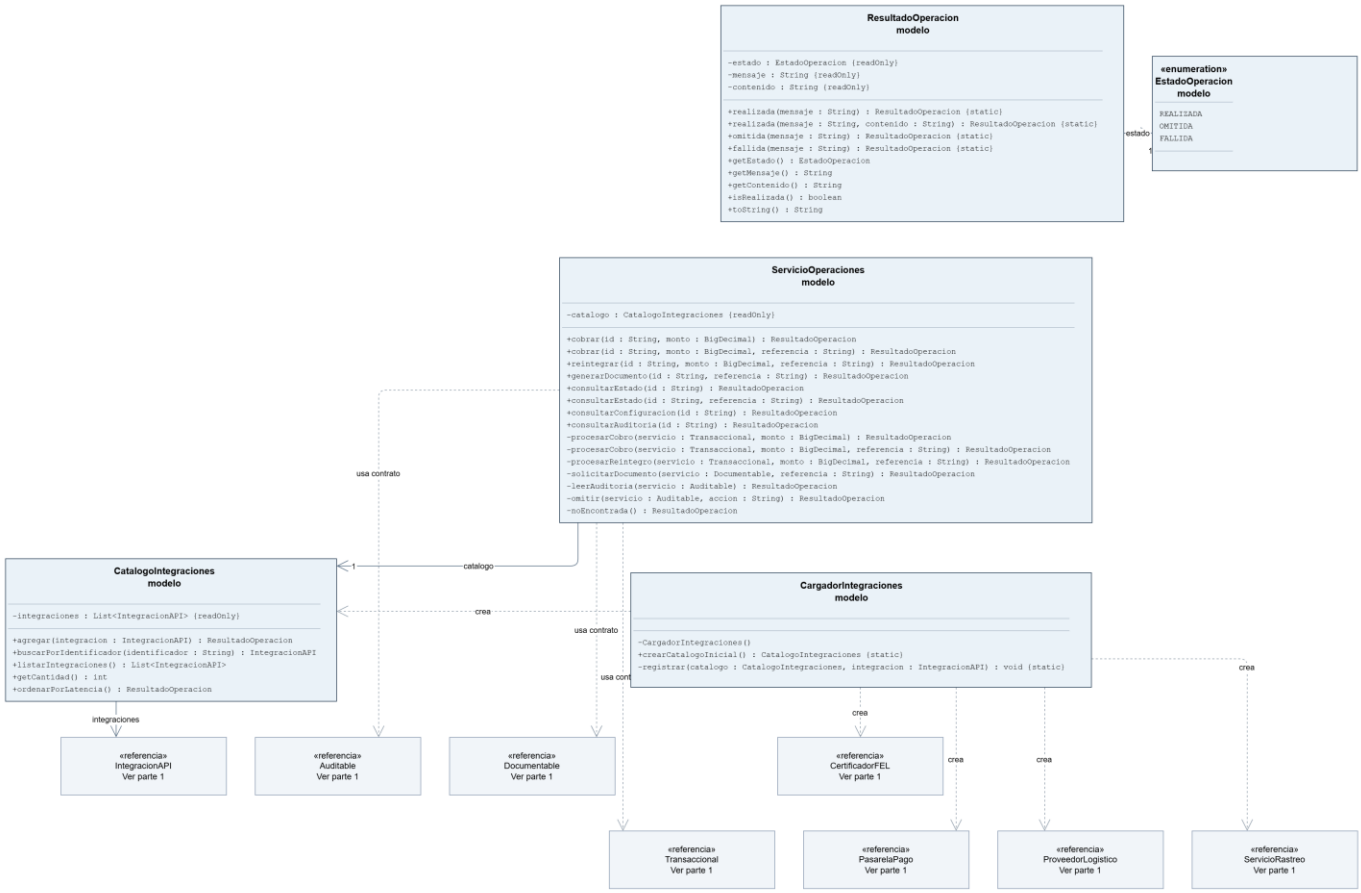
Diagrama UML:

Lo dividí en 3 partes porque el diagrama completo en una foto no se miraba bien.

Parte 1: modelo e interfaces

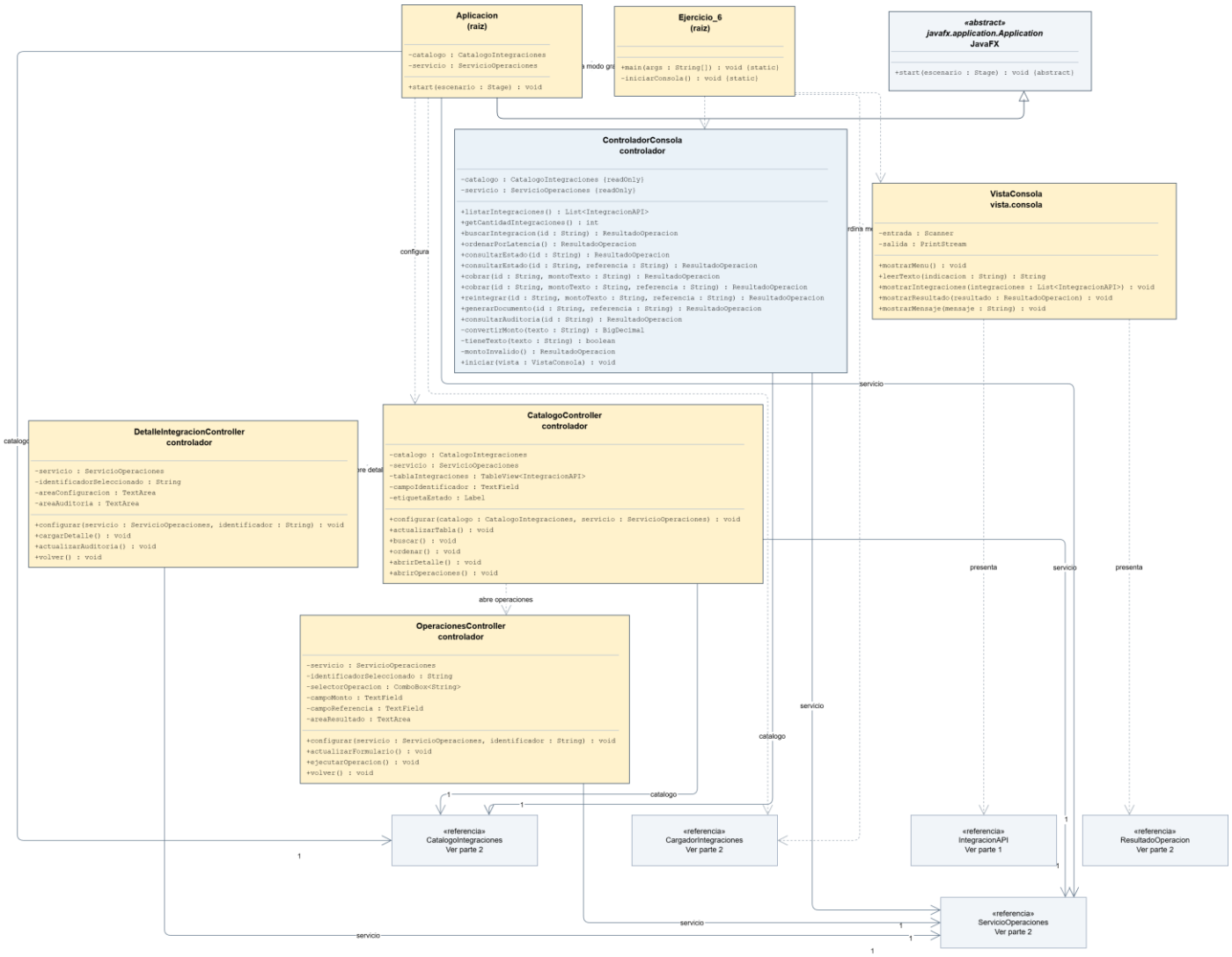


Parte 2: catálogo, operaciones y resultados



Una sola List<IntegracionAPI>. Los parametros Transaccional, Documentable y Auditable desacoplan el motor.
Las vistas JavaFX previstas son catalogo.fxml, detalle-integracion.fxml y operaciones.fxml; comparten estilos.css.

Parte 3: controladores, vistas y arranque


Aplicacion
(raiz)
-catalogo : CatalogoIntegraciones
-servicio : ServicioOperaciones
+start(escenario : Stage) : void
Ejercicio_6
(raiz)
+main(args : String[]) : void {static}
-iniciarConsola() : void {static}
«abstract»
javafx.application.Application
JavaFX
+start(escenario : Stage) : void {abstract}
ControladorConsola
controlador
-catalogo : CatalogoIntegraciones {readOnly}
-servicio : ServicioOperaciones {readOnly}
+listarIntegraciones() : List<IntegracionAPI>
+getCantidadIntegraciones() : int
+buscarIntegracion(id : String) : ResultadoOperacion
+ordenarPorLatencia() : ResultadoOperacion
+consultarEstado(id : String) : ResultadoOperacion
+consultarEstado(id : String, referencia : String) : ResultadoOperacion
+cobrar(id : String, montoTexto : String) : ResultadoOperacion
+cobrar(id : String, montoTexto : String, referencia : String) : ResultadoOperacion
+reintegrar(id : String, montoTexto : String, referencia : String) : ResultadoOperacion
+generarDocumento(id : String, referencia : String) : ResultadoOperacion
+consultarAuditoria(id : String) : ResultadoOperacion
-convertirMonto(texto : String) : BigDecimal
-tieneTexto(texto : String) : boolean
-montoInvalido() : ResultadoOperacion
+iniciar(vista : VistaConsola) : void
VistaConsola
vista.consola
-entrada : Scanner
-salida : PrintStream
+mostrarMenu() : void
+leerTexto(indicacion : String) : String
+mostrarIntegraciones(integraciones : List<IntegracionAPI>) : void
+mostrarResultado(resultado : ResultadoOperacion) : void
+mostrarMensaje(mensaje : String) : void
DetalleIntegracionController
controlador
-servicio : ServicioOperaciones
-identificadorSeleccionado : String
-areaConfiguracion : TextArea
-areaAuditoria : TextArea
+configurar(servicio : ServicioOperaciones, identificador : String) : void
+cargarDetalle() : void
+actualizarAuditoria() : void
+volver() : void
CatalogoController
controlador
-catalogo : CatalogoIntegraciones
-servicio : ServicioOperaciones
-tablaIntegraciones : TableView<IntegracionAPI>
-campoIdentificador : TextField
-etiquetaEstado : Label
+configurar(catalogo : CatalogoIntegraciones, servicio : ServicioOperaciones) : void
+actualizarTabla() : void
+buscar() : void
+ordenar() : void
+abrirDetalle() : void
+abrirOperaciones() : void
OperacionesController
controlador
-servicio : ServicioOperaciones
-identificadorSeleccionado : String
-selectorOperacion : ComboBox<String>
-campoMonto : TextField
-campoReferencia : TextField
-areaResultado : TextArea
+configurar(servicio : ServicioOperaciones, identificador : String) : void
+actualizarFormulario() : void
+ejecutarOperacion() : void
+volver() : void
«referencia»
CatalogoIntegraciones
Ver parte 2
«referencia»
CargadorIntegraciones
Ver parte 2
«referencia»
IntegracionAPI
Ver parte 1
«referencia»
ResultadoOperacion
Ver parte 2
«referencia»
ServicioOperaciones
Ver parte 2
configura
servicio
catalogo
abre operaciones
presenta
1